JC2001 Introduction to Software Engineering

Practical Session 8 - Exercises

Design Patterns

Design patterns are typical solutions to commonly occurring problems in software design. They are like pre-made blueprints that you can customise to solve a recurring design problem in your code. Design patterns are often confused with algorithms, because both concepts describe typical solutions to some known problems. While an algorithm always defines a clear set of actions that can achieve some goal, a pattern is a more high-level description of a solution. The code of the same pattern applied to two different programs may be different. In this exercise, we will work on various design patterns to solve general problems that software developers faced during software development.

Exercise A: Pattern Description

Most patterns are described very formally so engineers can reproduce them in many contexts. The sections that are usually present in a pattern description include:

- **Intent**: It briefly describes both the problem and solution.
- **Motivation**: It further explains the problem and the solution the pattern makes possible.
- **Structure**: Classes show each part of the pattern and how they are created.
- **Code example**: It makes it easier to grasp the idea behind the pattern.

The most universal and high-level patterns are *architectural patterns*. Developers can implement these patterns in virtually any language. Unlike other patterns, architectural patterns can be used to design the architecture of an entire application. Patterns can be categorised by their intent or purpose. There are three main groups of patterns:

- **Creational patterns**: They provide object creation mechanisms that increase flexibility and reuse of existing code.
- **Structural patterns**: They explain how to assemble objects and classes into larger structures
- while keeping these structures flexible and efficient.
- **Behavioural patterns**: They take care of effective communication and the assignment of responsibilities between objects.

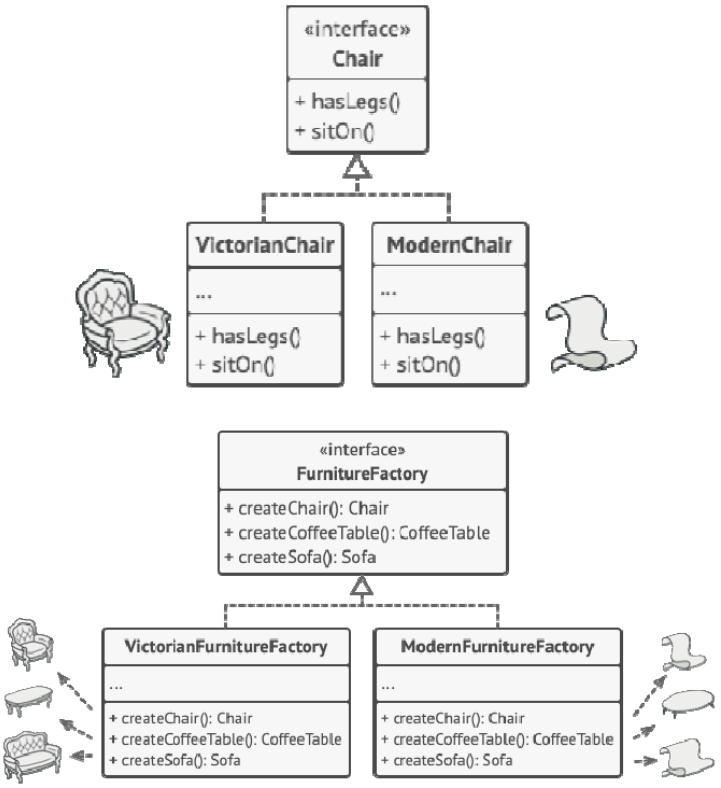
Question 1: What is the main difference between a design pattern and an algorithm?

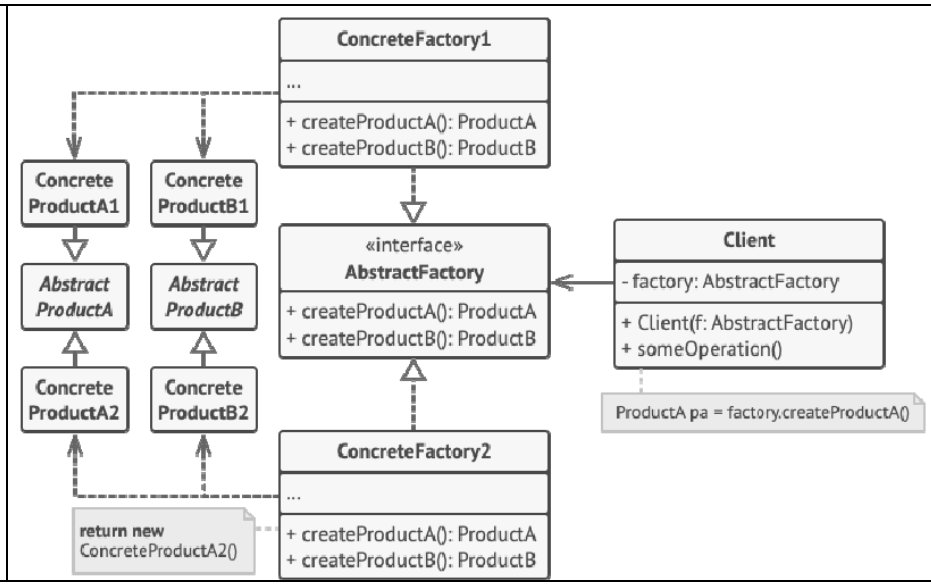
Question 2: Describe the benefits of design patterns in software development.

Exercise B: Creational Patterns

Creational design patterns provide various object creation mechanisms, which increase the flexibility and reuse of existing code. Creational patterns include (a) factory method, (b) abstract factory, (c) builder, (d) prototype and (e) singleton. We describe the abstract factory pattern below.

Abstract factory pattern	
Intent	The Abstract Factory is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.
Problem	You have a family of related objects and several variants of this family. As an example, chair, sofa and coffee table are a family of related products, and within this

	family the chair, sofa and coffee table are available in variants of modern, traditional and ancient styles. You need a way to create individual furniture objects so that they match other objects of the same family. You also want to be able to add new furniture to the catalog without changing existing code.
Solution	Explicitly declare interfaces for each distinct product of the product family. Then make all variants of products follow those interfaces. 
Structure	• **Abstract products** declare interfaces for a set of distinct but related products which make up a product family. • **Concrete products** are various implementations of abstract products, grouped by variants. Each abstract product must be implemented in all given variants. • The **Abstract factory** interface declares a set of methods for creating each of the abstract products. • **Concrete factories** implement creation methods of the abstract factory. Each concrete factory corresponds to a specific variant of products and creates only those product variants. • The **Client** can work with any concrete factory / product variant as long as it communicates with their objects via abstract interfaces.



Question 3: Describe the advantages and disadvantages of the Abstract factory pattern.

Question 4: You are a user-interface (UI) engineer in EFG software house. As part of your project, you will be implementing a user friendly graphical user interface. The client specifically asked for a crossplatform GUI, i.e., the GUI must work on Microsoft Windows, Macbook and Linux operating systems. The UI consists of buttons, checkboxes and drop boxes. Identify an appropriate design pattern and draw a class diagram for the cross-platform GUI.

References

1. Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides, *Design Patterns – Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1st edition, 1995. ISBN-13: 978-0201633610.
2. Design Patterns, https://refactoring.guru/design-patterns